

Presentation Script & Detailed Notes

Location-Based WebXR KML / KMZ Editor

Lab: WebXR Location-Based Project

Format: Speaker Notes & Q&A Guide

Speakers: Viola & Eric

Slide 1: Landing Page / Title

VIOLA

CARDS & BULLET POINTS ON SLIDE

- **Main Title:** Location-Based WebXR KML / KMZ Editor
- **Subtitle:** Browser-Based 3D Geodata Editing & Mobile Augmented Reality
- **Button:** Start Presentation

DETAILED EXPLANATION & SPEAKING NOTES

"Welcome to our final presentation for the WebXR Lab. Today we're presenting our browser-based Location-Based WebXR KML/KMZ Editor.

What is our project about? Classic KML and KMZ files (known from Google Earth) contain geographic data such as markers, paths, or 3D ground overlays. Until now, you needed specialized software or apps to view or edit this data. We have developed a purely web-based system that runs directly in the browser without any installation. It offers a two-part solution: a performant 3D desktop editor for convenient editing on a PC, and a mobile web app for smartphones that lets you see KML objects out in the field at their real GPS coordinates in augmented reality and move them with touch gestures."

Slide 2: 1. Introduction & Motivation

VIOLA

CARDS & BULLET POINTS ON SLIDE

- **Card 1 (The Goal & Key Advantage):** Browser-based 3D editor for KML/KMZ with no installation required. Lossless roundtrip editing & a dual system for desktop 3D & mobile AR.
- **Card 2 (Our Solution):** A dual system built on a shared Redux & GeoBridge foundation. Desktop editor for precision navigation & mobile AR for on-site visualization via GPS.
- **Card 3 (3 Screenshots):** Google Earth Source Map, 3D Desktop Editor, Mobile Field AR App.

DETAILED EXPLANATION & SPEAKING NOTES

"Let's take a closer look at our motivation and goal:

1. The problem with existing tools: *Anyone who wanted to use KML files on a smartphone or on the web previously ran into two problems: either there was no way to visualize them in AR at real GPS coordinates, or when saving edited files, vendor-specific tags (e.g., Google Earth styling information) were stripped out by standard XML parsers.*

2. Our solution (dual system): *We created an architecture that bridges two worlds:*

- *Desktop 3D GIS Editor: Enables convenient loading, viewing, and editing of KML objects on a PC in a Three.js 3D environment with OrbitControls, a property inspector, and undo/redo.*
- *Mobile WebXR AR App: Lets you take an AR-capable smartphone out into the field. The app uses the phone's camera and location sensors to render KML markers precisely at their real GPS coordinates in the real world.*

3. Core advantage (lossless roundtrip): *Every change made on the PC or on the smartphone is saved with absolutely no loss of data. That means: all original KML tags, comments, and formatting are 100% preserved when the file is later reopened in Google Earth."*

Slide 3: 2. System Architecture

VIOLA

CARDS & BULLET POINTS ON SLIDE

- **User / Device node:** Supplies GPS coordinates & phone compass data to the store + selects a KML file for `kmz-io`.
- `kmz-io`: Unpacks KMZ archives, reads in KML XML, and rebuilds ZIP files.
- **store (Redux):** Central state manager for document model, device state & command execution.
- `document-model` & `geo-bridge`: WGS84 geographic data conversion to 3D Cartesian meters.
- `editor` & `ar-scene`: Dual presentation layer (Desktop 3D vs. Mobile WebXR AR).

DETAILED EXPLANATION & SPEAKING NOTES

"Here you can see our modular system architecture. The diagram shows exactly how data flows through our system:

1. The user & the device (User / Device): At the top is the user. On desktop, they select a KML/KMZ file, which is passed to the `kmz-io` module. On the smartphone, the device additionally continuously supplies GPS position and compass orientation to the central `store`.

2. The core modules (`kmz-io` , `store` , `document-model`):

- `kmz-io` unpacks ZIP archives (KMZ) and passes the KML XML text on to the `document-model`.
- The `store` (built on Redux Toolkit) manages the entire application state centrally and deterministically.
- The `document-model` holds the abstract syntax tree (AST) — i.e., the XML tree of the KML file — in memory.

3. GeoBridge coordinate translation: The `document-model` stores geographic latitude and longitude values (WGS84). The `geo-bridge` converts these coordinates, relative to a local anchor origin point, into Cartesian 3D meters (x, y, z) for rendering in Three.js and WebXR — and vice versa when dragging objects!

4. The visualization layers (`editor` & `ar-scene`): Both the desktop editor and the mobile AR app access the same underlying data. This ensures that changes made in the editor are immediately reflected in the AR scene."

Slide 4: 2. Implementation Details

ERIC

CARDS & BULLET POINTS ON SLIDE

- **Card 1 (Lossless Roundtrip):** Targeted replacement of `<coordinates>` via AST substring mutation. Preserves 100% of all Google Earth styles, comments, and custom tags.
- **Card 2 (Geo-Bridge Mathematics):** Converts WGS84 (lat/lon/alt) into 3D meters (x,y,z) relative to an anchor origin point. Prevents floating-point precision errors on the GPU.
- **Code Snippet:** KML `<coordinates>` XML demonstration.

DETAILED EXPLANATION & SPEAKING NOTES

"Let's dive deeper into two important technical details:

1. How does the lossless roundtrip (loss-free editing) work?

*Ordinary XML parsers read in a file, build an object model, and regenerate the entire XML text when saving. In the process, formatting, comments, and unknown XML tags get lost. Our `document-model` instead uses **AST substring mutation**: when parsing, it records the exact character offset of the `<coordinates>` tags within the XML string. When the user moves a marker in 3D space, only the text inside that coordinates tag is swapped out. The rest of the XML code remains untouched, byte for byte. When you save the file and open it in Google Earth, all the original formatting is 100% preserved.*

2. The math behind the Geo-Bridge:

*Graphics cards (WebGL) compute in meters using 32-bit floating-point numbers. GPS coordinates, however, are degree values on the vast surface of the Earth. If you tried to convert global GPS numbers directly into 3D meters, the rendering would jitter heavily on the GPU. The `geo-bridge` solves this with a local tangent-plane coordinate system: the first GPS reading is fixed as the **session anchor $(0,0,0)$** . All KML objects are then calculated as relative meter distances (north, east, up) from this anchor. This guarantees millimeter-accurate, smooth rendering."*

Slide 5: 2. Implementation & Live Demo

ERIC

CARDS & BULLET POINTS ON SLIDE

- **Card 1 (Three.js Rendering Specs):** Supports 4 asset types: pins/sprites, 3D polylines, 2D ground overlays & 3D COLLADA (`.dae`) models. Includes GPU texture cache.
- **Live Demo Container (Right):** Embedded Three.js viewport (`renderers-demo`).

DETAILED EXPLANATION & SPEAKING NOTES

"In this slide you can see our rendering pipeline live in action.

1. Supported KML asset types:

Our Three.js renderers handle four different KML elements:

- *Pins / Sprites: 2D marker icons that always face the camera (billboarding).*
- *3D Polylines: Paths and routes rendered as 3D lines in space.*
- *2D GroundOverlays: Aerial images or maps projected precisely onto the ground grid.*
- *3D COLLADA models (`.dae`): Complete 3D buildings or objects, including textures.*

2. GPU texture cache & performance:

To avoid overloading the mobile device's memory, the system uses a central

`TexturePromiseCache` . If 100 identical markers are loaded, the image texture is uploaded to GPU memory only once and then reused.

(Here, on the slide, you can interact with the mouse in the embedded 3D window and show the various rendered objects.)"

Slide 6: 3. Architectural Insights (How It Works)

ERIC

CARDS & BULLET POINTS ON SLIDE

- **Step 1 (User Edit):** User drags a marker via drag-and-drop in desktop editor or touch gesture in AR.
- **Step 2 (Command Dispatch):** A `MoveMarkerCommand` is created and executed in Redux store (supports undo/redo).
- **Step 3 (Geo-Bridge Math):** GeoBridge converts the 3D meter displacement into new GPS coordinates.
- **Step 4 (AST Substring Edit):** Document model swaps out only the coordinates substring in KML XML.
- **Step 5 (Debounced Auto-Save):** Persistence waits 500ms after gesture ends before writing file to disk.
- **Key Takeaway:** Command pattern enables undo/redo & 500ms debouncing prevents stuttering during 60 FPS gestures.

DETAILED EXPLANATION & SPEAKING NOTES

"Let's walk step by step through what happens when the user edits an object:

Step 1 (User Edit): *The user drags a marker in the 3D editor, or presses and holds it on the phone screen and moves it.*

Step 2 (Command Dispatch): *The UI creates a `MoveMarkerCommand`. This command is executed in the Redux store. By encapsulating actions as command objects (command pattern), every action in the system is automatically reversible (Ctrl+Z / undo).*

Step 3 (Geo-Bridge Math): *The `geo-bridge` takes the changed 3D position (x, y, z) and calculates the new, exact WGS84 GPS coordinates (latitude and longitude) from it.*

Step 4 (AST Substring Edit): *The `document-model` finds the exact offset of the marker within the KML XML string and replaces only the text of the `<coordinates>` tag with the new coordinates.*

Step 5 (Debounced Auto-Save): *During a drag gesture running at 60 frames per second, continuously writing to disk would cause stuttering. That's why the persistence layer uses **500-millisecond debouncing**: only once the user releases the marker and 500ms have passed does the file get written to disk."*

Slide 7: 4. The Results (Desktop Editor Live Preview)

VIOLA

CARDS & BULLET POINTS ON SLIDE

- **Left Card (Desktop Editor Features):** Full 3D GIS editing in browser. OrbitControls camera navigation, object inspector, KML/KMZ upload & undo/redo.
- **Right Container:** Interactive full 3D editor (`editor-demo`).

DETAILED EXPLANATION & SPEAKING NOTES

"In this slide you can see the overall result of our desktop application, live.

The desktop editor offers a full 3D GIS experience in the web browser:

- *You can load any KML or KMZ file via drag-and-drop.*
- *You navigate freely around the objects in 3D using the mouse (OrbitControls).*
- *Clicking an object opens the inspector, where you can adjust its name, description, and GPS coordinates.*
- *Every change can be undone immediately.*

(Here, in the window on the right, you can load a KML file live or click and move an object.)"

Slide 8: 4. The Results (Mobile AR Field Demo Video)

VIOLA

CARDS & BULLET POINTS ON SLIDE

- **Left Card (Mobile AR Field Specs):** Mobile WebXR AR app. Live camera pass-through, real GPS vs. mock GPS toggle, grab-to-move touch editing directly in camera view.
- **Right Container:** Video player for phone screen recording (`ar-demo-video.mp4`).

DETAILED EXPLANATION & SPEAKING NOTES

"Here you can see the video of our mobile AR application being used in the field on a smartphone.

1. Camera pass-through & sensors:

When you open the app on an Android smartphone in the Chrome browser and step out into the field, the WebXR session starts. The live camera feed is displayed, and the framework fuses the phone's GPS data with the AR camera sensors.

2. Real GPS vs. mock GPS:

*For outdoor use, the app relies on the phone's real GPS. So that developers can also test the app at their desk, we built in a **mock GPS toggle** that lets you simulate location signals.*

3. Touch editing (grab-to-move):

Tapping a rendered KML marker on the phone screen lets you press and hold it directly in the camera view and drag it to a new spot in the field (grab-to-move). The new position is immediately converted into GPS coordinates and saved in the document."

Slide 9: 5. Problems Encountered

ERIC & VIOLA

CARDS & BULLET POINTS ON SLIDE

- **Card 1 (Workload & AI-Assisted Development):** High pace of development under heavy workload made it challenging to deeply understand every architectural layer.
- **Card 2 (AR Marker Positioning):** Viewer layer initially attached `featureGroup` directly to root scene instead of `arWorldGroup`, causing altitude & alignment drift.

DETAILED EXPLANATION & SPEAKING NOTES

"An honest look back at the challenges during the lab project:

1. Comprehensibility with a large amount of AI-generated code:

Because we developed a large amount of code with AI assistance under a heavy workload, it was a major challenge to immediately and deeply understand every single layer of the generated code (spanning more than 9 modules).

2. AR marker positioning in the viewer layer:

Toward the end, we ran into a problem with precise AR marker positioning on the phone: the viewer layer had attached the marker group (`featureGroup`) directly to the root scene instead of embedding it as a child of the framework's `arWorldGroup` . As a result, altitude elevation discrepancies (e.g. MSL KML height vs. ground level) buried markers underground. We fixed this by reparenting under `arWorldGroup` and applying local-floor altitude clamping (`y < 0 → y = 0`)."

Slide 10: 6. Lab Successes

ERIC

CARDS & BULLET POINTS ON SLIDE

- **Card 1 (Modular Prototyping):** Rapid implementation of a clean 9-component architecture using Redux & Three.js.
- **Card 2 (Lossless KML Editing):** 100% stable lossless roundtrip editing. Works flawlessly with Google Earth.

DETAILED EXPLANATION & SPEAKING NOTES

"Alongside the challenges, we achieved some important successes:

1. Modular architecture:

We structured the overall system into 9 fully independent, reusable modules (npm/Vite packages). Each module has its own unit tests and its own demo harness.

2. Reliable lossless editing:

Our core promise — lossless editing of KML files — works absolutely reliably. You can load a complex KML file from Google Earth, move markers in the browser, save it, and reopen the file in Google Earth: not a single style tag or comment is lost."

Slide 11: 7. Recap & Final Overview

ERIC & VIOLA

CARDS & BULLET POINTS ON SLIDE

- **Heading:** Recap: Key Takeaways
- **Screenshots & Diagram:** Architecture diagram + Google Earth + 3D Editor + Mobile AR App.

DETAILED EXPLANATION & SPEAKING NOTES

"Let's summarize the key points of our project:

- **No installation, on the web:** *Edit KML/KMZ files directly in the browser.*
- **Dual system:** *A precise 3D desktop GIS editor combined with a mobile WebXR AR app.*
- **100% lossless:** *AST substring mutation guarantees complete compatibility with Google Earth.*
- **Robust framework:** *Fusion of GPS, compass, and WebXR odometry for location-based AR."*

Slide 12: 7. Summary & Questions

ERIC & VIOLA

CARDS & BULLET POINTS ON SLIDE

- **Main Title:** Thank You for Your Attention!
- **Large Q&A Card:** Questions & Discussion (5 min).

DETAILED EXPLANATION & SPEAKING NOTES

"Thank you very much for your attention! We're now happy to take your questions."

Exhaustive Q&A Guide (Anticipated Questions & Detailed Answers)

Q1: Wie genau ist die Schnittstelle zwischen document-model und geo-bridge definiert – gibt es da einen klar dokumentierten Vertrag (Interface), oder ist das eng gekoppelt?

Antwort: Es gibt einen strikten, entkoppelten Vertrag über TypeScript-Interfaces

(`src/contracts/geo-bridge.ts` und `src/contracts/document-model.ts`). Das `document-model` kennt die `geo-bridge` gar nicht direkt, sondern speichert reine WGS84-Geokoordinaten. Die `geo-bridge` wird erst über die Edit-Commands (`ICommand.execute(doc, geoBridge)`) oder von den Renderern als reine Berechnungsfunktion hinzugezogen.

Q2: Warum wurden Desktop-Editor und Mobile-AR-App nicht als eine einzige responsive App gebaut, sondern als getrenntes Dual-System?

Antwort: Desktop-3D (Kamera-Navigation mit OrbitControls, Auswahltabellen, Datei-Uploads) und Mobile-AR (WebXR Immersive Session, Kamera-Stream, Sensor-Sensitivität, Touch-Grab-Gestik) haben völlig unterschiedliche UI-Paradigmen und Browser-APIs. Durch die Trennung in zwei schlanke Frontend-Sichten (`editor` und `ar-scene`), die beide auf denselben Redux-Store und dasselbe Dokumentenmodell zugreifen, konnte maximale Performanz und saubere Code-Struktur erreicht werden.

Q3: Was passiert bei gleichzeitigen Änderungen an mehreren Koordinaten – verschieben sich die gespeicherten Offsets, wenn eine frühere Ersetzung die String-Länge ändert?

Antwort: Jedes Command arbeitet atomar auf dem DOM/AST-Baum des `document-model`. Wenn ein Koordinaten-String durch eine Ersetzung länger oder kürzer wird (z. B. durch geänderte Nachkommastellen), aktualisiert das `document-model` intern die betroffenen XML-Offsets bzw. führt die Ersetzung auf Knotenebene durch. Zudem sorgt `formatCoordinate` in der GeoBridge für stabile Nachkommastellen, um unnötige Zeichenlängen-Schwankungen zu verhindern.

Q4: Funktioniert Undo/Redo geräteübergreifend (z. B. eine AR-Änderung am Handy per Desktop rückgängig machen), oder nur lokal pro Gerät/Session? Wie wird der Command-Stack bei App-Neustart behandelt?

Antwort: In der aktuellen Phase1/It.1 ist der Command-Stack (Undo/Redo) in-memory im Redux-Store des jeweiligen Geräts gekoppelt. Beim Speichern wird das finale KML/KMZ-Dokument persistiert. Der Undo-Stack ist also lokal pro Session aktiv. Bei App-Neustart wird das persistierte Dokument geladen; der Undo-Stack startet frisch für die neue Bearbeitungssitzung.

Q5: Was ist der Vorteil eures Systems gegenüber bestehenden Tools wie Google Earth Web oder spezialisierten GIS-Programmen (z. B. QGIS)? Wo ist die Nische?

Antwort: 1. **Nichts zu installieren (Pure Web):** Läuft direkt im mobilen Browser ohne App-Store-Installation. 2. **Lossless Roundtrip:** Standard-GIS-Tools werfen oft herstellerspezifische Google-Earth-Tags oder Formatierungen. Unser AST-Substring-Ansatz garantiert 100 % Kompatibilität. 3. **Feld-AR mit Echtzeit-GPS:** Direkte AR-Visualisierung und Dragging vor Ort an der echten GPS-Position gibt es weder in QGIS noch in Google Earth Web.

Q6: Was wäre der nächste Schritt, um das System produktionsreif zu machen?

Antwort: 1. Multi-User-Synchronisation über WebSockets / CRDTs, um simultanes Editieren im Feld und im Büro zu ermöglichen. 2. High-Accuracy RTK-GPS Anbindung für zentimetergenaue Vermessung im Feld. 3. Erweiterung der 3D-Kachelung (3D Tiles / Terrain Elevation APIs) für gebirgiges Gelände im Desktop-Editor.